

3.1.1. Numpy Array Operations

08:25

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`.
- The shape of the array using `shape`.
- The total number of elements in the array using `size`.

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, `r` and `c`, representing the number of rows and the number of columns.
- The next `r` lines contain `c` space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

numpyarr...

```
1 import numpy as np
2 r, c = map(int, input().split())
3 elements = []
4 for _ in range(r):
5     row = list(map(int, input().split()))
6     elements.extend(row)
7 arr = np.array(elements).reshape(r, c)
8 print(arr)
9 print(arr.ndim)
10 print(arr.shape)
11 print(arr.size)
```

Average time

0.011 s

11.50 ms

Maximum time

0.019 s

19.00 ms

2 out of 2 shown test case(s) passed

10 out of 10 hidden test case(s) passed

Test case 1

Expected output

Actual output

Terminal

Test cases

< Prev Reset Submit Next >

3.2.1. Numpy: Matrix Operations

03:00

The given code takes two 3×3 matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. Addition (`matrix_a + matrix_b`)
2. Subtraction (`matrix_a - matrix_b`)
3. Element-wise Multiplication (`matrix_a * matrix_b`)
4. Matrix Multiplication (`matrix_a . matrix_b`)
5. Transpose of Matrix A

Input Format:

- The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

matrixOp...

```
11 # Addition
12 print("Addition (A + B):")
13 print(matrix_a + matrix_b)
14 # Subtraction
15 print("Subtraction (A - B):")
16 print(matrix_a - matrix_b)
17 # Multiplication (element-wise)
18 print("Element-wise Multiplication (A * B):")
19 print(matrix_a * matrix_b)
20 # Matrix multiplication (dot product)
21 print("A dot B:")
22 print(np.dot(matrix_a, matrix_b))
23 # Transpose
24 print("Transpose of A:")
25 print(matrix_a.T)
```

Average time

0.016 s

16.00 ms

Maximum time

0.019 s

19.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Terminal

Test cases

< Prev Reset Submit Next >

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases +

stacking.py

```
1 range(3)]]
2
3
4
5
6
7 print("Enter Array2:")
8 arr2 = np.array([list(map(int, input().split())) for i in
9 range(3)])
10
11
12 # Perform horizontal stacking (hstack)
13 h_stack = np.hstack((arr1, arr2))
14 print("Horizontal Stack:")
15 print(h_stack)
16
17 # Perform vertical stacking (vstack)
18 v_stack = np.vstack((arr1, arr2))
19 print("Vertical Stack:")
```

Average time

0.019 s

19.00 ms

Maximum time

0.024 s

24.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 24 ms

Debug

Terminal

Test cases

< Prev Reset Submit Next >

3.2.3. Numpy: Custom Sequence Generation

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using numpy based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Sample Test Cases +

customS...

```
1 import numpy as np
2
3 # Take user input for the start, stop, and step of the sequence
4 start = int(input())
5 stop = int(input())
6 step = int(input())
7
8 # Generate the sequence using np.arange()
9 sequence = np.arange(start, stop, step)
10 # Print the generated sequence
11 print(sequence)
```

Average time

0.009 s

8.75 ms

Maximum time

0.011 s

11.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 11 ms

Debug

Expected output

Actual output

Terminal

Test cases

< Prev Reset Submit Next >

3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Op...

You are given two arrays `A` and `B`. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations

- Calculate the mean, median, and standard deviation of array `A`.

3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: `A1 OR B1`).

Input Format:

- The first line contains space-separated integers representing the elements of array `A`.
- The second line contains space-separated integers representing the elements of array `B`.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the

different...

```
1 import numpy as np
2
3 def array_operations(A, B):
4
5     # Convert A and B to NumPy arrays
6     A = np.array(A)
7     B = np.array(B)
8
9     # Arithmetic Operations
10    sum_result = A + B
11    diff_result = A - B
12    prod_result = A * B
```

Average time

0.010 s

Maximum time

0.016 s

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 18 ms Debug

Terminal

Test cases

< Prev Reset Submit Next >

3.2.5. Numpy: Copying and Viewing Arrays

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

```
Original array after modifying view: <original_array>
View array: <view_array>
```

- After modifying the copy:

```
Original array after modifying copy: <original_array>
```

copyAnd...

```
1 import numpy as np
2
3 inputlist = list(map(int, input().split(" ")))
4
5 # Original array
6 original_array = np.array(inputlist)
7
8 # Create a view
9 view_array = original_array.view()
10
11 # Create a copy
12 copy_array = original_array.copy()
```

Average time

0.006 s

Maximum time

0.009 s

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 9 ms Debug

Terminal

Test cases

< Prev Reset Submit Next >

3.2.5. Numpy: Copying and Viewing Arrays

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

```
Original array after modifying view: <original_array>
View array: <view_array>
```

- After modifying the copy:

```
Original array after modifying copy: <original_array>
```

copyAnd...

```
1 import numpy as np
2
3 inputlist = list(map(int, input().split(" ")))
4
5 # Original array
6 original_array = np.array(inputlist)
7
8 # Create a view
9 view_array = original_array.view()
10
11 # Create a copy
12 copy_array = original_array.copy()
```

Average time

0.005 s

5.75 ms

Maximum time

0.009 s

9.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 9 ms Debug

Terminal

Test cases

< Prev Reset Submit Next >